Static Keyword in Java :

The static keyword in Java is used for memory management and belongs to the class rather than any specific instance. It allows members (variables, methods, blocks, and nested classes) to be shared among all objects of a class.

- Memory is allocated only once when the class is loaded.
- No object creation is needed to access static members; use the class name directly.
- Static methods and variables can't access non-static members directly.
- Static methods can't be overridden because they belong to the class, not instances.

1️⃣ Where Does static Live in Memory?

When a Java program runs, JVM creates different memory areas:

🔹 **1. Heap Memory**

- Stores **objects**
- Every time you use new, memory is allocated in heap.

🔹 **2. Stack Memory**

- Stores **method calls, local variables**
- Each thread has its own stack.

🔹 **3. Method Area (Metaspace in Java 8+)**

- Stores:
  - Class metadata
  - Static variables
  - Method bytecode
  - Runtime constant pool

👉 **Static members are stored in the Method Area (Metaspace)** — not in Heap.

2️⃣ What Happens When a Class Loads?

When JVM loads a class:

Step 1: Class Loading

- Class bytecode is loaded into memory.

Step 2: Linking

- Memory allocated for static variables.
- Default values assigned.

Step 3: Initialization

- Static blocks execute.
- Static variables get assigned actual values.

Example:

```
class Demo {
  static int x = 10;

  static {
     System.out.println("Static block executed");
  }
}
```

When Demo is loaded:

- Memory for x is allocated.
- Static block runs once.
- x is stored in Method Area.

Even if you create 100 objects:

```
Demo d1 = new Demo();
Demo d2 = new Demo();
```

👉 x exists **only once** in memory.

3️⃣ Why Static is for Memory Management

Without static:

```
class Student {
  String college = "ABC";
}
```

If 1000 objects are created:

- 1000 copies of college are stored in Heap.

With static:

```
class Student {
  static String college = "ABC";
}
```

Now:

- Only ONE copy exists in Method Area.
- All objects refer to same memory.

✅ Saves memory
✅ Avoids duplication

4️⃣ Static Variables – Internal Working

```
class Counter {
  static int count = 0;

  Counter() {
     count++;
  }
}
```

Memory Flow:

1. Class loads → count created in Method Area.

2. Each object creation increments same count.
3. No separate copy per object.

So:

Counter c1 → count = 1
Counter c2 → count = 2
Counter c3 → count = 3

Only ONE count.

---

5 Static Methods – Why No Object Needed?

```
class MathUtils {
  static int add(int a, int b) {
     return a + b;
  }
}
```

Call:

```
MathUtils.add(10, 20);
```

Because:

- Static methods belong to class.
- JVM does not require object reference.
- Stored in Method Area.

---

6 Why Static Cannot Access Non-Static Directly?

Non-static members live in Heap (inside objects).

Static members live in Method Area.

So static method:

```
class Test {
```

```
    int x = 10;

    static void show() {
        System.out.println(x); // ❌ error
    }
}
```

Problem:

- Static method runs without object.
- x belongs to object.
- No object reference available.

Solution:

```
static void show() {
    Test t = new Test();
    System.out.println(t.x);
}
```

7️⃣ Static Block – Why & When Used?

```
class Config {
    static {
        System.out.println("Load DB config");
    }
}
```

Runs:

- Only once
- When class is loaded
- Before main() if class is referenced

Used for:

- Database driver loading
- Configuration setup
- Static initialization

Example (real world):

Class.forName("com.mysql.cj.jdbc.Driver");

Driver registers via static block internally.

---

8 Static Method Cannot Be Overridden – Why?

Overriding is runtime polymorphism.
Static methods use compile-time binding.

Example:

```
class Parent {
   static void show() {
       System.out.println("Parent");
   }
}

class Child extends Parent {
   static void show() {
       System.out.println("Child");
   }
}
```

Now:

```
Parent obj = new Child();
obj.show();
```

Output:

Parent

Because:

- Method call resolved at compile time.
- Based on reference type, not object type.
- This is called **method hiding**, not overriding.

9️⃣ Static Nested Class – Memory Behavior

```
class Outer {
  static class Inner {
     void display() {
        System.out.println("Static nested class");
     }
  }
}
```

Why useful?

- Does not need outer class object.
- Saves memory.
- No implicit reference to outer instance.

🔟 Important JVM Detail (Java 8+)

Before Java 8:

- Static variables stored in Permanent Generation (PermGen).

After Java 8:

- Stored in **Metaspace**
- Metaspace uses native memory (not heap).
- Grows dynamically.

1️⃣ Default Initialization (Automatic)

When a class loads, JVM allocates memory for static variables in **Method Area (Metaspace)** and assigns default values.

Data Type	Default Value
int	0
double	0.0
boolean	false
Object	null

Example:

```java
class Demo {
    static int x;
    static String name;

    public static void main(String[] args) {
        System.out.println(x);     // 0
        System.out.println(name);  // null
    }
}
```

✔ No manual initialization needed.

2️⃣ Initialize at Declaration

```java
class Demo {
    static int x = 10;
}
```

This happens during **class initialization phase**.

4️⃣ Initialize Later (After Class Load)

Yes, you can assign later:

```java
class Demo {
    static int x;
}

public class Test {
    public static void main(String[] args) {
        Demo.x = 100;
    }
}
```

✔ This is allowed.

But remember:

- Memory already allocated.
- You're just updating value.

Find **duplicate elements** from this list: ['A', 'B', 'C', 'A', 'D', 'E', 'B']

✅1️⃣ Using Set (Simple & Efficient) – O(n)

```java
import java.util.*;

public class DuplicateExample {
  public static void main(String[] args) {

      List<String> list = Arrays.asList("A","B","C","A","D","E","B");

      Set<String> seen = new HashSet<>();
      Set<String> duplicates = new HashSet<>();

      for (String value : list) {
         if (!seen.add(value)) {
            duplicates.add(value);
         }
      }

      System.out.println(duplicates);
  }
}
```

Output:

[A, B]

Using Stream + Set (Short & Clean)

```
 List<String> list = Arrays.asList("A","B","C","A","D","E","B");
Set<String> seen = new HashSet<>();

Set<String> duplicates = list.stream()
      .filter(e -> !seen.add(e))
      .collect(Collectors.toSet());
```

—----------

🌐 **What is a Cookie?**

A **Cookie** is small data stored in the **browser (client side)**.

- Stored in user's browser
- Sent automatically with every request to same domain
- Maximum size ~4KB
- Can expire

Example:
When you login to a website, server may send a cookie like:

Set-Cookie: userId=101; Max-Age=3600

Browser stores it and sends it back in every request.

Create Cookie in Servlet

```java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class CookieExample extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        Cookie cookie = new Cookie("username", "Ramesh");
        cookie.setMaxAge(60 * 60); // 1 hour
        response.addCookie(cookie);

        response.getWriter().println("Cookie Created");
    }
}
```

To Read Cookie:

```java
Cookie[] cookies = request.getCookies();

for (Cookie c : cookies) {
    if (c.getName().equals("username")) {
        System.out.println(c.getValue());
    }
}
```

new Cookie("username", "Ramesh");

- "username" → Key (name)
- "Ramesh" → Value

So internally it behaves like:

username = Ramesh

When browser stores it, it looks like:

username=Ramesh

What Extra Things Does Cookie Object Have?

Cookie is not just key-value. It also contains:

- name
- value

- maxAge
- domain
- path
- secure flag
- httpOnly flag

Example:

```
cookie.setMaxAge(3600);
cookie.setHttpOnly(true);
cookie.setSecure(true);
```

So internally, object looks like:

```
Cookie Object:
{
  name: "username",
  value: "Ramesh",
  maxAge: 3600,
  httpOnly: true,
  secure: true
}
```

---

4 Memory View

When you write:

```
Cookie cookie = new Cookie("username", "Ramesh");
```

Memory:

Heap:

```
Cookie Object
  name → "username"
  value → "Ramesh"
  maxAge → default (-1)
```

Stack:

cookie → reference to heap object

5️⃣ What Is Actually Sent to Browser?

When you do:

response.addCookie(cookie);

Server converts object into HTTP header:

Set-Cookie: username=Ramesh; Max-Age=3600

Browser only sees key=value format.

It does NOT know about Java object.

—----------

🌐 What is a Session?

A **Session** stores data on the **server side**.

- Stored in server memory
- Each user has unique session
- Session ID is stored in cookie (usually JSESSIONID)
- More secure than cookies

So technically:

👉 Session uses cookie internally to store session ID.

🧠 How Session Works Internally

1. User sends first request.
2. Server creates session object.

3. Server generates unique session ID.
4. Server sends session ID in cookie (JSESSIONID).
5. Browser sends that cookie in next requests.
6. Server identifies user using session ID.

🔹 Create Session in Servlet

Java

```java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class SessionExample extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession();  // create session

        session.setAttribute("username", "Ramesh");

        response.getWriter().println("Session Created");
    }
}
```

To Read Session:

Java

```java
HttpSession session = request.getSession(false);

if (session != null) {
    String user = (String) session.getAttribute("username");
    System.out.println(user);
}
```

1️⃣ Is Session an Object?

Yes ✅

HttpSession is an **interface** from:

javax.servlet.http.HttpSession

The server (like Tomcat) provides implementation.

When you call:

HttpSession session = request.getSession();

- A **session object is created in server memory (heap)**.
- Each user gets a unique session.
- It has a unique session ID.

2️⃣ Does Session Store Key–Value?

Yes ✅

When you do:

session.setAttribute("username", "Ramesh");

It stores data as:

key → "username"
value → "Ramesh"

Internally it works like a Map:

Map<String, Object>

So session is:

👉 An object
👉 That stores key-value pairs
👉 On server side

3️⃣ Memory View

When session is created:

Server Memory (Heap)

```
Session Object
  id → "ABC123XYZ"
  attributes → Map
      username → "Ramesh"
      role → "Admin"
  creationTime
  lastAccessTime
```

Browser

Browser only stores:

JSESSIONID = ABC123XYZ

Browser DOES NOT store actual data.

---

4 How Session Works Step-by-Step

1. User sends request.
2. Server creates session object.
3. Server generates session ID.
4. Sends cookie:

   Set-Cookie: JSESSIONID=ABC123XYZ

5. Browser stores that.
6. Next request → browser sends:

   Cookie: JSESSIONID=ABC123XYZ

7. Server finds correct session object.

—----------

Delete character at index from specific string:

```
String str = "banana";
String result = str.replace("a", ""); // Because you are replacing "a" with an empty string ""

System.out.println(result); // bnn
```

✅2️⃣ Using StringBuilder (Delete)

Here you can actually **delete**.

- **Delete character at index**

```
StringBuilder sb = new StringBuilder("banana");
sb.deleteCharAt(2);

System.out.println(sb);  // baana
```

- **Delete range**

```
StringBuilder sb = new StringBuilder("banana");
sb.delete(1, 4);  // removes index 1 to 3

System.out.println(sb);  // bna
```

—----------

Why Can't we use Java 7 in Spring Boot ?

📌 Java Version Compatibility with Spring Boot

Spring Boot Version	Minimum Java Version
1.x	Java 7 (older versions only)
2.x	Java 8
3.x	Java 17

- **If You Are Using:**

✅ **Spring Boot 3.x**

- Requires **Java 17**
- Uses **Jakarta EE**
- Java 7 ❌ Not supported

✅ **Spring Boot 2.x**

- Requires **Java 8 minimum**
- Java 7 ❌ Not supported

⚠️ **Spring Boot 1.x (very old)**

- Some early versions supported Java 7
- But it is **deprecated and not recommended**
- No security updates

🚨 **Why Java 7 Is Not Supported?**

Spring Boot uses:

- Lambda expressions
- Streams API
- Optional
- Default methods in interfaces

These features came in **Java 8**.

Modern libraries like:

- **Spring Framework 6**
- **Resilience4j**
- Latest MySQL drivers

All require Java 8+.

—----------

1️⃣ What is @EnableAutoConfiguration?

@EnableAutoConfiguration tells Spring Boot:

> "Based on the dependencies present in the classpath, automatically configure the required beans."

Example

If you add:

spring-boot-starter-web

Spring Boot automatically configures:

- DispatcherServlet
- Embedded Tomcat
- Jackson ObjectMapper
- Spring MVC configuration

You don't need to configure them manually.

2️⃣ Where is it used?

Usually you don't write it directly.

It is already included inside:

@SpringBootApplication

And @SpringBootApplication is a combination of:

@Configuration

@EnableAutoConfiguration

@ComponentScan

So when you use:

```
@SpringBootApplication
public class MyApp {
  public static void main(String[] args) {
      SpringApplication.run(MyApp.class, args);
  }
}
```

Auto configuration is already enabled.

3️⃣ How does @EnableAutoConfiguration work internally?

It works using:

```
@EnableAutoConfiguration
 ↓
@Import(AutoConfigurationImportSelector.class)
 ↓
Reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
 ↓
Loads configuration classes conditionally
```

Spring Boot checks:

- Is a specific class present?
- Is a bean already defined?
- Is a property enabled?

It uses conditional annotations like:

- @ConditionalOnClass
- @ConditionalOnMissingBean
- @ConditionalOnProperty
- @ConditionalOnWebApplication

Example

DataSourceAutoConfiguration runs only if:

- JDBC dependency exists
- No custom DataSource bean exists

4️⃣ How to Exclude Specific Auto Configuration?

Sometimes you don't want certain auto configuration.

✅ Method 1: Exclude in @SpringBootApplication

```
@SpringBootApplication(
  exclude = {DataSourceAutoConfiguration.class}
)
public class MyApp {
}
```

✅ Method 2: Exclude using @EnableAutoConfiguration

```
@EnableAutoConfiguration(
  exclude = {DataSourceAutoConfiguration.class}
)
```

✅ Method 3: Exclude using application.properties

```
spring.autoconfigure.exclude=\
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

6️⃣ Real-World Example

Case: You added JPA dependency but don't want DB config

Spring tries to configure:

- DataSource
- Hibernate
- JPA

If DB properties are missing → application fails.

So you exclude:

```
@SpringBootApplication(
  exclude = {
    DataSourceAutoConfiguration.class,
    HibernateJpaAutoConfiguration.class
  }
)
```

Now app starts without DB.

—----------

Normally in **Spring Boot**,
if **any bean creation fails**, the application **will not start**.

Because Spring's IoC container must successfully create all required beans at startup.

In **Spring Boot**, by default:

> ❌ If any bean creation fails → Application startup fails.

Spring is **fail-fast** by design.

But in some scenarios (optional beans, external systems down, feature toggles), you may want the application to start even if some beans fail.

🚨 First Important Thing

You **cannot force Spring Boot to start** if:

- A required bean fails
- A dependency is missing
- A circular dependency occurs
- A @Bean throws exception during initialization

Spring will stop with:

APPLICATION FAILED TO START

✅ But… You CAN ignore or handle bean errors using these techniques:

1️⃣ Make Bean Optional (Recommended) : @ConditionalOnProperty is used for conditional bean loading based on configuration properties

Use @ConditionalOnProperty

```
@Bean
@ConditionalOnProperty(name = "feature.payment.enabled", havingValue = "true")
```

```
public PaymentService paymentService() {
  return new PaymentService();
}
```

In application.properties:

```
feature.payment.enabled=false
```

👉 Bean will not load
👉 Application starts successfully

✅ What It Does

It tells Spring:

> "Create this bean ONLY IF a specific property is present and matches a value."

If condition is false → bean is completely **ignored** during startup.

🔎 How Spring Internally Works

When Spring Boot starts:

1. Reads application.properties
2. Loads Environment
3. Processes configuration classes
4. Evaluates conditional annotations
5. Decides whether to register bean in IoC container

If condition fails →
Spring **does not register the bean at all**

So:

- No bean
- No initialization
- No error
- Application starts normally

🧠 **Example**

```
@Bean
@ConditionalOnProperty(
    name = "feature.payment.enabled",
    havingValue = "true",
    matchIfMissing = false
)
public PaymentService paymentService() {
  return new PaymentService();
}
```

application.properties

```
feature.payment.enabled=false
```

What Happens?

- Property value = false
- Condition expects true
- Bean is skipped
- App starts successfully

2️⃣ @Lazy (Detailed Explanation)

✅ What It Does

Normally Spring creates all singleton beans at startup.

But when you use:

```
@Service

@Lazy

public class PaymentService {

}
```

Spring does NOT create it during startup.

Instead:

- It creates a proxy
- Actual object is created only when first used

🔎 Internal Working

Without @Lazy:

Startup → Bean Created → Constructor runs → Init runs

With @Lazy:

Startup → Proxy created → Real bean NOT created
When method is called → Real bean created

🧠 **Example**

```
@Service

@Lazy

public class PaymentService {

  public PaymentService() {

      System.out.println("PaymentService Created");

  }

}
```

If never used → constructor never runs.

—----------

It is part of **Java 8** (java.util.concurrent package).

By default, it is used for **non-blocking async execution**, but it can also behave synchronously.

🔹 1️⃣ **Asynchronous (Default Use Case)**

```
CompletableFuture<String> future =
  CompletableFuture.supplyAsync(() -> "Hello");

future.thenAccept(System.out::println);
```

- Runs in background thread
- Main thread continues
- Non-blocking

🔹 2️⃣ **Synchronous Behaviour (Blocking)**

You can make it synchronous using:

✅ **get()**

✅ **join()**

```
CompletableFuture<String> future =
  CompletableFuture.supplyAsync(() -> "Hello");

String result = future.join();  // BLOCKS until complete
System.out.println(result);
```

Here:

- Main thread waits
- Acts like synchronous call

So yes ✅ it supports synchronous behaviour by blocking.

—----------

Find the department which has highest average salary using Java 8 Streams

To **find the department with the highest average salary using Java 8 Streams**, let's assume you have an Employee class like this:

```
class Employee {
  private String name;
  private String department;
  private double salary;

  // constructor
  public Employee(String name, String department, double salary) {
     this.name = name;
     this.department = department;
     this.salary = salary;
  }
```

```
  public String getDepartment() { return department; }
  public double getSalary() { return salary; }
}
```

✅ Step 1: Sample Data

```
List<Employee> employees = Arrays.asList(
  new Employee("Ramesh", "IT", 60000),
  new Employee("Suresh", "IT", 80000),
  new Employee("Mahesh", "HR", 50000),
  new Employee("Naresh", "HR", 70000),
  new Employee("Rajesh", "Finance", 90000)
);
```

```
// Group employees by department and calculate average salary
Map<String, Double> avgSalaryByDept = employees.stream()
   .collect(Collectors.groupingBy(
      Employee::getDepartment,
      Collectors.averagingDouble(Employee::getSalary)
   ));
```

✅ Step 1: Find Department & Average Salary

```
Map<String, Double> avgSalaryByDept =
      employees.stream()
            .collect(Collectors.groupingBy(
                  Employee::getDepartment,
                  Collectors.averagingDouble(Employee::getSalary)
            ));

System.out.println(avgSalaryByDept);
```

👉 Output Example:

```
{IT=70000.0, HR=60000.0, Finance=90000.0}
```

Now we have department + average salary.

```
String dep = avgSalaryByDept .entrySet()
.stream()
.max(Map.Entry.comparingByValue())
.map(Map.Entry::getKey)
.orElse(null);
```

Or

```
String dep = avg.entrySet()
```

```
Optional<Map.Entry<String, Double>>
highestAvgDeptOpt = avgSalaryByDept.entrySet()
   .stream()
   .max(Map.Entry.comparingByValue());

highestAvgDeptOpt.ifPresent(entry ->
   System.out.println("Department: " + entry.getKey() +
                  " (Avg Salary: " + entry.getValue() + ")"));
```

```
    .stream()
    .collect(Collectors.maxBy(
        Comparator.comparingDouble(e -> e.getValue()))
    .map(e -> e.getKey())
    .orElse(null);
```

—----------

—----------

The @Component annotation in Spring is a stereotype annotation used to mark a Java class as a Spring-managed bean. In simple terms, it tells the Spring framework: "Please detect this class during component scanning and create an object (bean) of it in the application context."

🔹 1️⃣ @Component

✅ What is it?

@Component is a **class-level annotation** used to tell Spring:

> "This class is a Spring-managed bean. Automatically detect and create it."

It works with **component scanning**.

📌 Example

```
@Component
public class PaymentService {

  public void pay() {
    System.out.println("Payment done");
  }
}
```

When Spring starts:

- It scans the package
- Finds @Component
- Creates object of PaymentService
- Stores it in Spring container

🔹 **How it works internally?**

When you use:

@ComponentScan("com.example")

Spring scans that package and automatically registers all classes annotated with:

- @Component
- @Service
- @Repository
- @Controller

These are called **stereotype annotations** (they are specialized versions of @Component).

✅ **When to use @Component?**

✔ When you control the class source code
✔ For your own business classes
✔ For service, repository, controller classes

🔹 2️⃣ **@Bean**

✅ **What is it?**

@Bean is a **method-level annotation** used inside a @Configuration class.

It tells Spring:

> "Call this method and register the returned object as a bean."

📌 **Example**

@Configuration

```
public class AppConfig {

  @Bean
  public PaymentService paymentService() {
     return new PaymentService();
  }
}
```

Here:

- Spring calls paymentService()
- The returned object is registered as a bean

🔹 **How it works internally?**

@Configuration class is processed by Spring.
Every method annotated with @Bean:

- Is executed
- The return object is stored in Spring container

🔥 Key Differences

Feature	@Component	@Bean
Type	Class-level annotation	Method-level annotation
Used for	Your own classes	Third-party or manual object creation
Scanning needed?	Yes (Component Scan)	No
Control	Less control	Full control (you create object)

—----------

Method Reference

In Java, a Method Reference is a **short way to refer to an existing method** instead of **writing a lambda expression**. It is used mainly with **functional interfaces in Streams or functional programming.**

Instead of writing a lambda like this:

```
list.forEach(s -> System.out.println(s));
```

You can use a **method reference**:

```
list.forEach(System.out::println);
```

System.out::println refers to the existing println method.

So **method reference = shorthand for lambda when the lambda only calls an existing method.**

Syntax

ClassName::methodName

Example:

```
String::length
System.out::println
Math::sqrt
```

Types of Method References

1. Reference to a Static Method

A static method reference is used to refer to a static method of a class. It replaces a lambda expression that simply calls a static method.

- Uses class name to access the method
- Method must be static
- Commonly used with utility methods

Syntax:

ClassName::staticMethodName

Syntax:

ClassName::staticMethodName

```java
import java.util.*;

class MathUtil{
  static void square(int n) {
     System.out.println(n * n);
  }
}

class GFG{
  public static void main(String[] args) {

     List<Integer> list = Arrays.asList(2, 3, 4);
     list.forEach(MathUtil::square);
  }
}
```

Output

```
4
9
16
```

Explanation:

- square(int n) is a static method that prints the square of a number.
- forEach(MathUtil::square) passes each list element to the square() method.
- Internally, this works like:

list.forEach(n -> MathUtil.square(n));

2. Reference to an Instance Method of a Particular Object

This type of method reference refers to an instance method of a specific object. The object is already created, and its method is referenced directly.

- Uses an existing object
- Method must be non-static
- Suitable for object-specific operations

Syntax:

objectReference::instanceMethod

```
import java.util.*;

class Printer{
  void print(String msg) {
      System.out.println(msg);
  }
}

class GFG{
  public static void main(String[] args) {

      Printer printer = new Printer();
      List<String> data = Arrays.asList("Java", "Spring", "Boot");

      data.forEach(printer::print);
  }
}
```

Output

```
Java
Spring
Boot
```

Explanation:

- printer is an object of the Printer class.
- printer::print refers to the instance method print().
- Each element of the list is automatically passed to print().
- Equivalent lambda expression:

data.forEach(msg -> printer.print(msg));

3. Reference to Instance Method of Arbitrary Object

This method reference refers to an instance method of any object of a particular class. The actual object is determined at runtime.

- Uses class name instead of object
- Mostly used with Stream API
- Method is called on each element

Syntax:

ClassName::instanceMethod

```
import java.util.*;

class GFG{

  public static void main(String[] args){

    List<String> names = Arrays.asList("java", "spring", "microservice");

    names.stream()
        .map(String::toUpperCase)
        .forEach(System.out::println);
  }
}
```

Output

JAVA
SPRING
MICROSERVICE

Explanation:

- String::toUpperCase calls toUpperCase() on each String object.
- map() transforms each element into uppercase.
- System.out::println prints the transformed values.
- Equivalent lambda expression:

map(name -> name.toUpperCase())

4. Reference to a Constructor

A constructor reference is used to create a new object using a functional interface. It replaces a lambda expression that calls a constructor.

- Uses new keyword
- Commonly used with Supplier, Function
- Helps in object creation logic

Syntax:

ClassName::new

```
import java.util.function.Supplier;

class Student {
  Student() {
    System.out.println("Student object created");
  }
}

class GFG{
  public static void main(String[] args) {

    Supplier<Student> supplier = Student::new;
    supplier.get();
  }
}
```

Output

Student object created

Explanation:

- Supplier<Student> is a functional interface with get() method.
- Student::new refers to the constructor of Student.
- When supplier.get() is called, a new Student object is created.

- Equivalent lambda expression:

Supplier<Student> supplier = () -> new Student();

—----------

Optional class

1 Create Optional from a String

If a String may be null, use **Optional.ofNullable()**.

```
import java.util.Optional;

String str = null;

Optional<String> opt = Optional.ofNullable(str);
```

---

2 Provide a Default Value

Use **orElse()** if the value is null.

```
String result = Optional.ofNullable(str)
                .orElse("Default Value");

System.out.println(result);
```

Output:

Default Value

---

3 Execute Code Only if Value Exists

Use **ifPresent()**.

```
Optional<String> opt = Optional.ofNullable(str);

opt.ifPresent(s -> System.out.println(s));
```

This runs **only if the value is not null**.

—----------

Assume that you have a list of names and your list contains a mix of names that are ending with "lal" and not ending with lal as well.

1) Your code must list the names ending with "lal".

For eg if the list of names is {Ram,Vinitha,Mohanlal,Jawaharlal} your code must display only Mohanlal and Jawaharlal.

2)Your code must display the names not ending with "lal" - it must display only Ram and Vinitha from above list.

3)Add null as input to your list i.e. {Ram,Vinitha,Mohanlal,Jawaharlal,null} - your code must not throw null pointer exception and as well display names not ending with "lal".

Case 1: Display names ending with "lal"

```
import java.util.Arrays;
import java.util.List;

public class Case1 {
  public static void main(String[] args) {

      List<String> names = Arrays.asList("Ram","Vinitha","Mohanlal","Jawaharlal");

      for(String name : names){
         if(name.endsWith("lal")){
            System.out.println(name);
         }
      }
  }
}
```

Output

Mohanlal
Jawaharlal

Case 2: Display names not ending with "lal"

```
import java.util.Arrays;
import java.util.List;

public class Case2 {
  public static void main(String[] args) {

      List<String> names = Arrays.asList("Ram","Vinitha","Mohanlal","Jawaharlal");

      for(String name : names){
         if(!name.endsWith("lal")){
            System.out.println(name);
         }
      }
  }
}
```

Output

Ram
Vinitha

Case 3: Handle null input without NullPointerException

```
import java.util.Arrays;
import java.util.List;

public class Case3 {
  public static void main(String[] args) {

      List<String> names = Arrays.asList("Ram","Vinitha","Mohanlal","Jawaharlal", null);

                 for(String name : names){
                if(name != null && !name.endsWith("lal")){
```

```
                System.out.println(name);
            }
        }
    }
}
```

Output

Ram
Vinitha

Key Point

name != null check prevents **NullPointerException** when calling endsWith().